Power Platform DevOps Structure

Best Practices and Strategies for Effective DevOps Transformation

Document Information

Version	Issue Date	Changes	Author
1.0		Created By	Divyani Rajani

Table of Contents

1. Executive Summary 4
2. Current State Assessment 4
2.1 Existing Process 4
2.2 Challenges 4
3. Target State Vision 4
3.1 Core Objectives 4
3.2 Sprint Structure – 2 Weeks 4
4. Team Roles and Responsibilities 6
5. Work Item Management 6
5.1 Work Item Types & Iteration Assignment 6
5.2 Iteration and Carryover 7
5.3 Epic & Feature Management 7
5.4 Story (Development, Maintenance, Enhancement), Task & Bug Status Flow 8
5.5 Purpose of all Work Item Statuses 9
5.6 Work Item Type Hierarchy 10
5.7 Story Points Estimation 12
6. Request Intake, Review, and Approval Workflow 12
7. Agile Work Item Lifecycle and Deployment Flow 15
8. Agile Best Practices & Work Artifacts 15
9. Environments – Dev, QA, UAT & Prod 16
10. Tooling and Documentation 16
11. Success Metrics 16
12. The Importance of Your Contribution 17

DevOps Structure Guide with Agile Integration for Power Platform

1. Executive Summary

This document outlines the restructuring of the current DevOps process to incorporate Agile methodology, with a focus on Power Platform solutions. The goal is to implement a structured, iterative, and collaborative process that promotes visibility, efficiency, and quality in delivery.

2. Current State Assessment

2.1 Existing Process

The current development and deployment processes involve a significant degree of manual effort and are only partially standardized. Solution packages are still manually exported and imported across environments, and this approach will remain in place until deployment pipelines are fully established. Although a monthly sprint planning cycle has been adopted, Agile practices are not fully implemented or consistently followed throughout the development lifecycle. Documentation is limited, resulting in weak traceability of changes and decisions. Testing is performed on an ad hoc basis, with no formalized strategy or test automation in place, impacting the overall reliability of quality assurance.

2.2 Challenges

Deployments are inconsistent due to the absence of an automated pipeline and the lack of a structured deployment timeline, although they are generally not error prone. This makes it difficult to coordinate releases and ensure alignment across environments. While sprint planning is conducted monthly, there is limited collaboration and alignment across teams, which impacts the effective delivery of planned work. Additionally, there is no established feedback loop or regular retrospective practice, reducing opportunities for continuous improvement and shared learning.

3. Target State Vision

3.1 Core Objectives

- ✓ Agile development processes will be implemented using structured 2-week sprint cycles to enhance planning, execution, and delivery cadence.
- ✓ Clearly defined roles and responsibilities will be established to ensure accountability and improve collaboration across teams.
- ✓ A structured manual deployment approach will be introduced to bring consistency to solution deployments until deployment pipelines are in place.
- ✓ Continuous testing and feedback mechanisms will be embedded throughout the sprint cycle to support early issue identification and iterative improvement.
- ✓ Governance practices will be introduced incrementally, ensuring process standardization while maintaining flexibility to adapt to evolving business needs.

3.2 Sprint Structure – 2 Weeks

The sprint cycle is structured over two weeks, with built-in flexibility for iterative development and early testing. Test deployments occur as soon as development is complete, enabling rapid feedback from internal teams. User Acceptance Testing (UAT) follows test deployments, allowing business/ Site users to validate functionality against requirements and provide feedback before release. Production deployments are scheduled for Wednesday in Week 1 of the following sprint, ensuring proper UAT coverage, release readiness, and team availability.

Daily Stand-Up

A short, time-boxed meeting where each team member shares:

- What they worked on yesterday
- What they plan to do today
- Any blockers

Goal: Keep the team aligned and surface issues early.
Attendees: Development Team, Business Analyst, Scrum Master, Product Owner

Backlog Refinement

Focus on reviewing and prioritizing items in the product backlog. Ensures:

- Tickets are clearly defined
- Acceptance criteria are complete
- Priority is set

Goal: Prepare tickets for upcoming sprint grooming and estimation.
Attendees: Product Owner, Scrum Master, Business Analyst

Sprint Review & Planning

- Sprint Review: Team demonstrates completed work to stakeholders and collects feedback.
- Sprint Planning: Define sprint goal and select backlog items for next sprint.

Attendees: Product Owner, Scrum Master, Development Team, QA/Testers, Business Analyst

Development Freeze

Marks the cut-off point for introducing new features or changes in the sprint.

- Only critical bug fixes are allowed after this point.

Ensures stability for testing and production release.

Release Communication Prep

- Finalize release notes and deployment documentation
- Draft communications for end-users
- Align all stakeholders on release plan

Goal: Ensure smooth rollout and clarity on what is being delivered.

Retrospective

- Reflect on the sprint: successes, challenges, and areas of improvement
- Identify actionable changes for future sprints

Attendees: Scrum Master, Product Owner, Development Team, QA/Test Team

Sprint Deployment Readiness / Prod Deployment

- Validate deliverables meet quality standards
- Confirm deployment readiness

- Execute production rollout
- Communicate status to stakeholders

Attendees: Release Manager, Development Team, QA/Test Team, Product Owner

This rhythm establishes a **sustainable 2-week delivery cycle** with business-aligned release predictability.

4. Team Roles and Responsibilities

Role	Responsibilities
Product Owner	Prioritize backlog, approve user stories, validate business needs.
Scrum Master	Facilitate Agile ceremonies, remove blockers, ensure adherence to process.
Tech Leads	Support backlog grooming, assign tickets to developers, provide technical guidance, perform production deployments, and mentor the team.
Developers	Deliver features, perform unit testing, deploy to QA.
Testers/QA	Design test cases, validate solutions, support UAT.
Release Manager	Govern release readiness, coordinate production deployments, track approvals.
Business Analyst	Refine requirements, support backlog grooming, bridge business and development teams.
Comms/Change/Training	Manage release notes, communications, and training needs.

5. Work Item Management

5.1 Work Item Types & Iteration Assignment

Type	Description	Iteration Assignment	Notes
New Request	Intake of a new requirement	N/A	Captured in backlog; reviewed for feasibility.
Epic	Strategic initiative spanning multiple sprints	Not bound to iteration	Epics are not assigned to a sprint but can have child Features or work items linked to iterations.
Feature	Functional grouping of work items	Not bound to iteration	Features are linked to Epics; child items (Development, Maintenance, Enhancement, Tasks) may be assigned to iterations.
Development	New functionality development	Assigned to sprint	Captures new dev work; may move to next sprint if not completed.

Maintenance	Break-fix, access changes, or operational upkeep of existing functionality	Assigned to sprint	Ensures existing systems continue to operate as expected; includes urgent fixes or routine maintenance.
Enhancement	Improvements or upgrades to existing features	Assigned to sprint	Adds new capabilities or improves existing functionality; may span multiple sprints if dependent on other work.
Task	Technical step required to deliver a work item	Assigned to sprint	Supports Development, Maintenance, or Enhancement; can span sprints if needed.
Bug	Defect or issue found in QA/UAT	Assigned to sprint	Critical bugs prioritized for immediate fix; others scheduled in backlog.
Blocker	An issue that prevents progress on other work items	Assigned to sprint (if tied to active work)	Tracked separately for visibility; resolution is prioritized to unblock dependent items.

5.2 Iteration and Carryover

- Development, Maintenance, and Enhancement work items are assigned to sprints.
- If a work item cannot be completed within a sprint, it is reviewed during sprint planning and either:
    1. Moved to the next sprint iteration, or
    2. Split into new work items with updated assignments and iteration.
- **Ensure that all tickets are kept up to date with the latest status and details.**

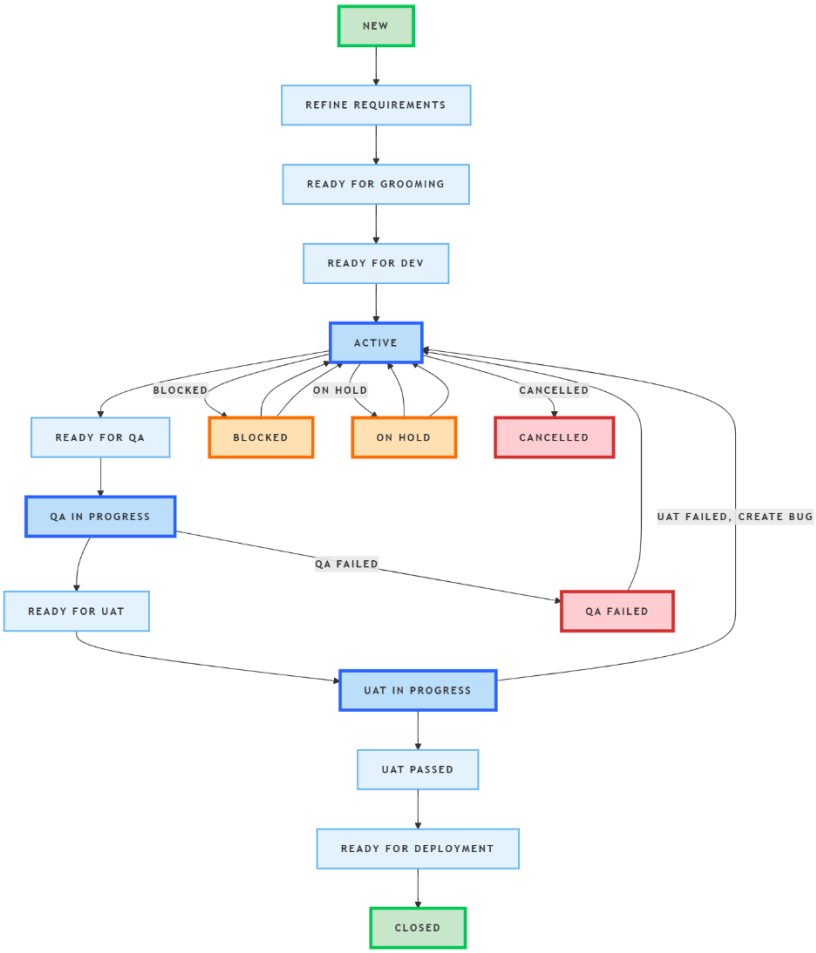
5.3 Epic & Feature Management

- Epics and Features are not assigned to iterations or individual team members.
- Child Work Items: Development, Maintenance, and Enhancement items linked to Epics/Features are assigned to specific iterations. Snapshots track updates and progress.

5.4 Story (Development, Maintenance, Enhancement), Task & Bug Status Flow

Primary Flow:

- **Alternate Paths:** QA Failed, Blocked, On Hold, Cancelled



5.5 Purpose of all Work Item Statuses

To streamline tracking and promote clarity, the following statuses are defined for each work item type:

New Request

Status	Purpose
New	When a new request is received
Intake form sent	Intake form sent to gather high level requirements
More info required	When we need addition information regarding the request
Waiting for approval	Waiting for approval on the request
Intake form received	When we have received the complete intake form
Ready for feasibility check	When the request is ready to be viewed by Tech leads for feasibility check
Feasible	When the request is doable
Completed	When the work item has been created and added to backlog
Not feasible	When the request is not doable in our environment
On Hold	Work temporarily paused due to internal reasons
Cancelled	Work item is no longer valid or will not be pursued further.

Epic & Feature Status

State	Purpose
Development	Indicates that work on the Epic/Feature is in progress. Although Epics and Features are not assigned to specific iterations, this state reflects active development.
Released	Marks that the Epic/Feature has been fully delivered. Once a new Epic or Feature is implemented and released, it should be moved to this state.
Inactive	Used when an Epic/Feature is no longer being pursued or is out of scope.

Development, Maintenance, Enhancement, Task & Bugs Work Item Status

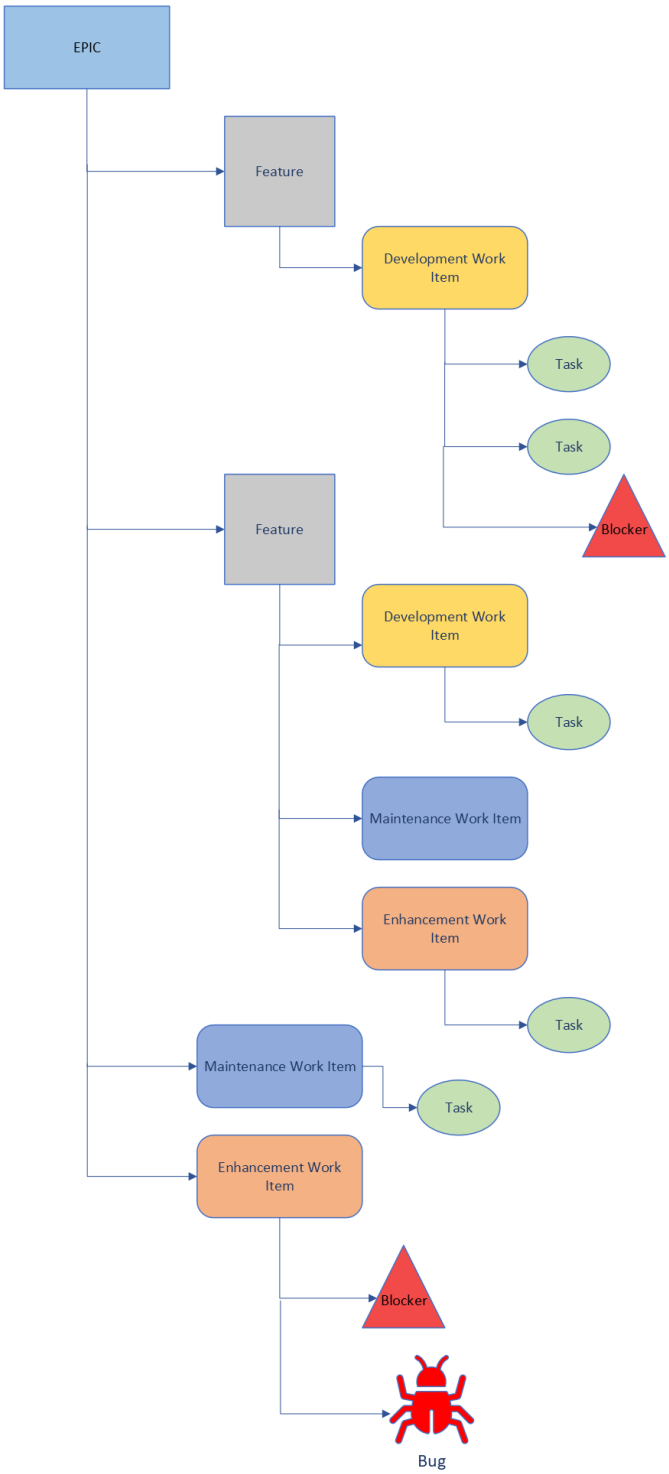
State	Purpose
New	Work item created and added to backlog.
Refine requirements	Business Analyst (BA) is clarifying and detailing requirements.
Ready for Grooming	Ready for prioritization and story pointing (estimation).
Ready for Dev	Waiting for a developer to start active work.
Active	Development or work is actively in progress.
Ready for QA	Development/work completed; testing team can start QA.
QA In Progress	Manual or automated testing is ongoing.
QA Failed	Bugs or issues found during testing that need resolution.
Ready for UAT	QA complete; work item is ready for User Acceptance Testing.
UAT In Progress	Business users or stakeholders are performing acceptance testing.
UAT Passed	UAT completed successfully; item meets business acceptance.
Checklist In Progress	Status shows the deployment checklist is being worked on
Ready for Deployment	Item is ready to be deployed to production.
On Hold	Work temporarily paused due to internal reasons.
Blocked	Progress halted due to external dependencies or blockers.
Closed	Work item is complete, tested, accepted, and deployed.
Cancelled	Work item is no longer valid or will not be pursued further.

Blocker Status

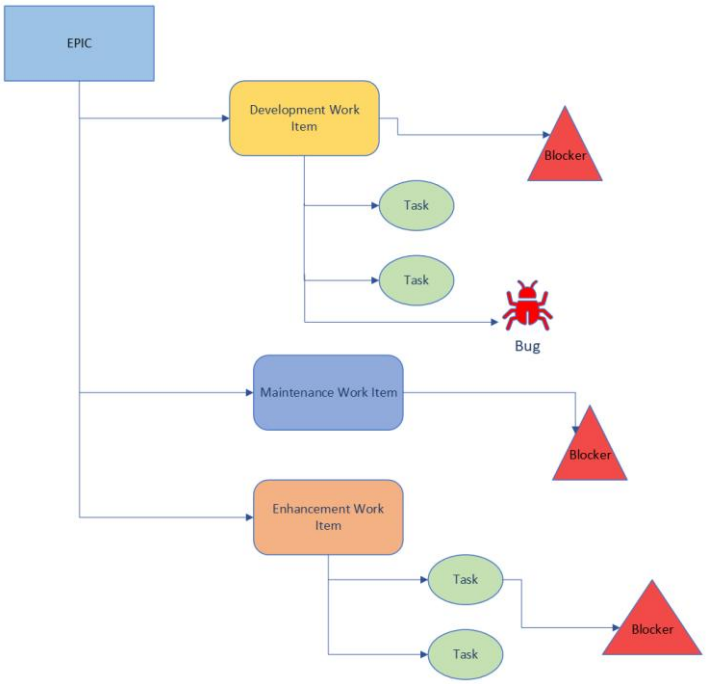
State	Purpose
Active	Blocker is currently being investigated or worked on. It is impacting progress and needs immediate attention.
Closed	Blocker has been resolved and cleared; normal workflow can resume.
Cancelled	Blocker is no longer valid or will not be addressed further.

5.6 Work Item Type Hierarchy

POWER APPS


EPIC
Feature
Development Work Item
Task
Task
Blocker
Feature
Development Work Item
Task
Maintenance Work Item
Enhancement Work Item
Task
Maintenance Work Item
Task
Enhancement Work Item
Blocker
Bug

POWERBI



5.7 Story Points Estimation

To standardize effort estimation, the team will use **story points** to represent the relative complexity and effort required to complete a work item.

- **Estimation Scale:**
  - 1 Story Point = ~0.5 day of effort
  - Story points should be estimated during backlog grooming and sprint planning.
- **Maximum Story Size:**
  - If a story is estimated to exceed **10 story points**, it must be broken down into smaller, manageable tasks.
  - This ensures that work items are achievable within a sprint and provide better visibility on progress.

Using this approach will help the team plan sprints more effectively, maintain predictable delivery, and avoid oversized stories that could block sprint completion.

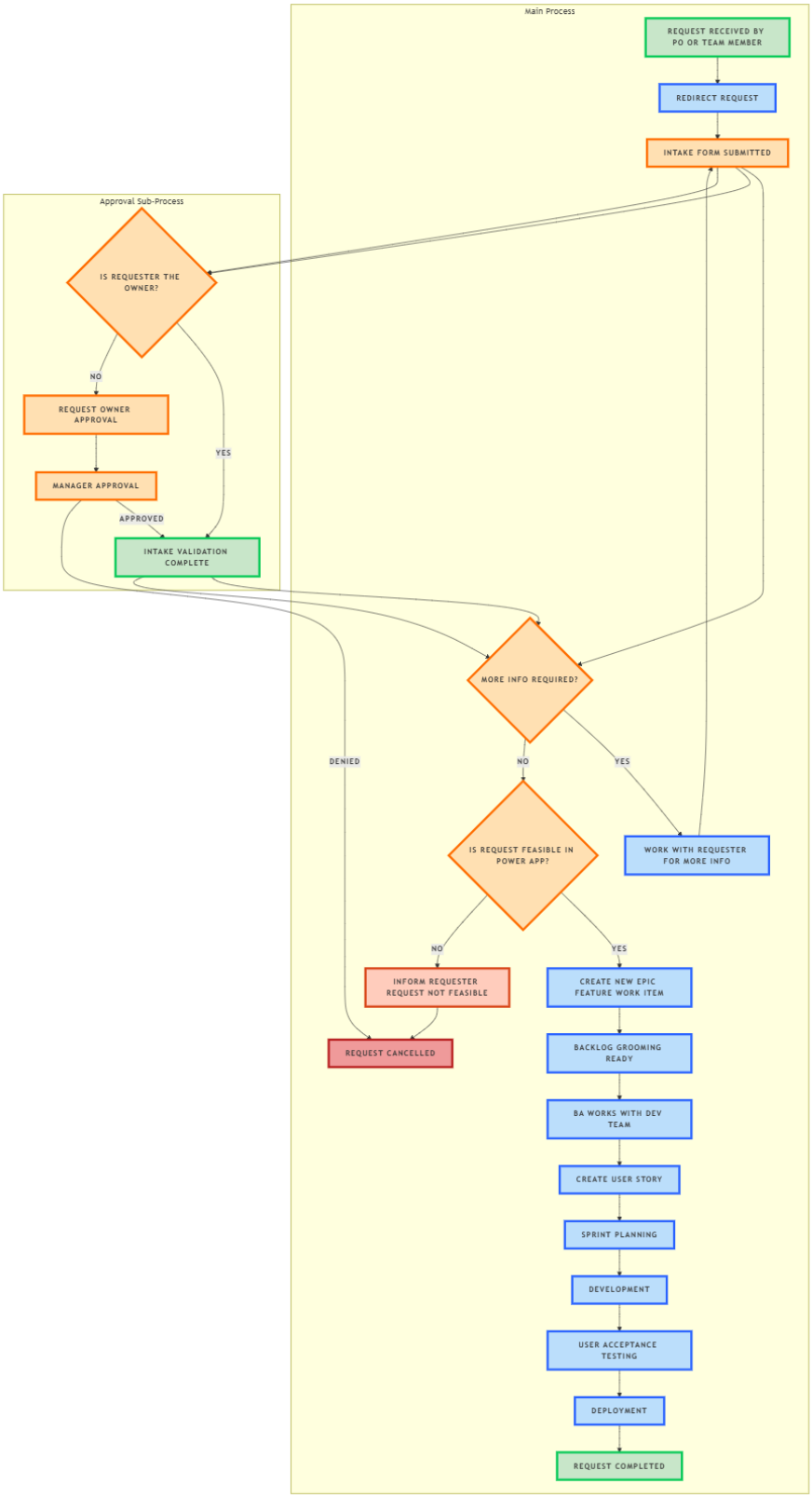
6. Request Intake, Review, and Approval Workflow

- All requests (new, enhancement, or maintenance) are formally logged through an intake form and reviewed by the PO/BA.

- Requests undergo an approval process and a feasibility check (with Tech Lead & BA) before being added to the DevOps backlog.

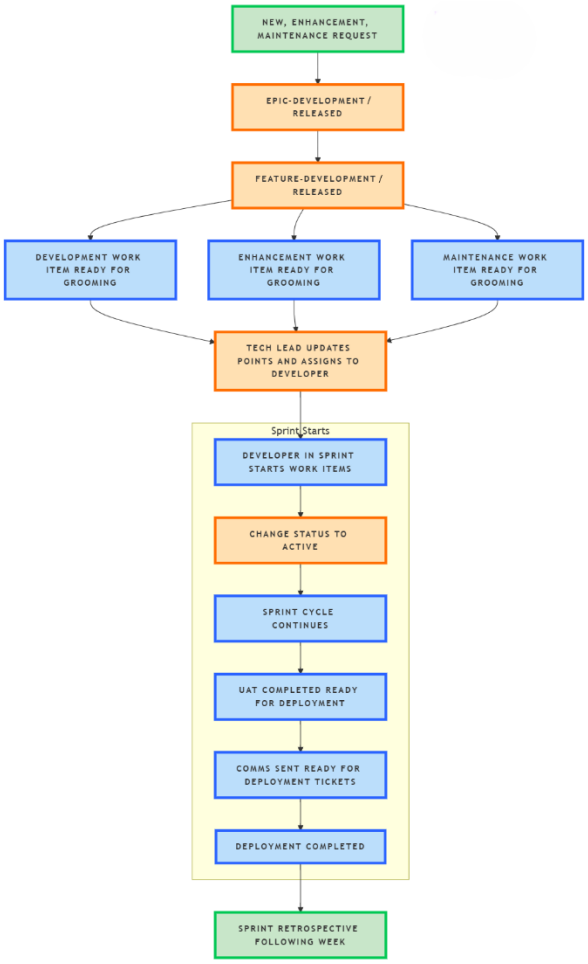
- Approved items are refined, groomed, and prioritized for sprint planning, then executed through Development → QA/ UAT → Deployment.

- A structured approval sub-process ensures governance: non-owners must obtain approval from the Request Owner or Manager, with rejections notified promptly.


Main Process
REQUEST RECEIVED BY PO OR TEAM MEMBER
REDIRECT REQUEST
INTAKE FORM SUBMITTED
Approval Sub-Process
IS REQUESTER THE OWNER?
NO
YES
REQUEST OWNER APPROVAL
MANAGER APPROVAL
APPROVED
INTAKE VALIDATION COMPLETE
MORE INFO REQUIRED?
DENIED
NO
YES
IS REQUEST FEASIBLE IN POWER APP?
WORK WITH REQUESTER FOR MORE INFO
NO
YES
INFORM REQUESTER REQUEST NOT FEASIBLE
CREATE NEW EPIC FEATURE WORK ITEM
REQUEST CANCELLED
BACKLOG GROOMING READY
BA WORKS WITH DEV TEAM
CREATE USER STORY
SPRINT PLANNING
DEVELOPMENT
USER ACCEPTANCE TESTING
DEPLOYMENT
REQUEST COMPLETED

7. Agile Work Item Lifecycle and Deployment Flow

Below diagram illustrates the end-to-end flow of handling new requests, enhancements, and maintenance items within the Agile delivery process. Work begins at the Epic and Feature levels, then moves into specific work items (Development, Enhancement, or Maintenance) that are groomed and assigned by Tech Leads. Developers execute these items within the sprint, progressing through active development, UAT, and deployment readiness. Communications are shared prior to deployment, and the cycle concludes with deployment completion and a sprint retrospective.



8. Agile Best Practices & Work Artifacts

Naming Standards

- All apps, reports, and work items should follow proper naming conventions to ensure consistency and easier tracking. The format for Development, Maintenance, and Enhancement work item types should follow the "<Who> wants <what>[, <why>]" format, where the "why" is optionally included to explain the business value of the request.
- The "who wants what, why" format also helps to ensure that the work items are broken down into right sized pieces, where each ticket represents a single piece of functionality.

9. Environments – Dev, QA, UAT & Prod

Development (DEV)

- Active coding, feature development, and unit testing occur here.
- Developers follow proper naming standards for apps, reports, and artifacts.
- Updates are captured for documentation and traceability.
- Managed solutions are exported from DEV to QA/UAT for testing.
- Continuous integration practices applied to ensure solutions are ready for promotion.

QA (TEST) / UAT

- Managed solutions from DEV are imported for testing.
- Full QA cycles are executed, including functional, regression, and integration testing.
- UAT validations are performed with business stakeholders.
- Continuous deployments occur during the sprint to test incremental changes.
- Bugs are tracked, prioritized, and fixed before PROD deployment.

Production (PROD)

- Stable environment accessible to business users.
- Deployments take place on Wednesday of Week 1 in the next sprint, following the development freeze.
- Only approved features/changes from QA/UAT are deployed.
- Deployment requires:
    - UAT approval
    - Rollback plan in case of issues
    - Release communication prep for end-users and stakeholders
- Post-deployment validation ensures deliverables meet business and quality standards.

10. Tooling and Documentation

Effective tooling enables coordination, transparency, and traceability throughout the delivery process. The following tools support backlog management, collaboration, and documentation:

- ✓ Backlog Management: Azure DevOps Boards – used for stories, sprint tasks, and work tracking.
- ✓ Documentation: SharePoint, Wiki, or OneNote – used for maintaining architecture decisions, solution design and testing evidence.
- ✓ Communication: Microsoft Teams & Outlook – used for daily team collaboration, status updates, and stakeholder communication.

11. Success Metrics

The following KPIs (Key Performance Indicators) will be used to monitor progress, identify areas for improvement, and measure the success of Agile adoption:

- **Sprint Completion Rate**: Percentage of committed work completed within the sprint timeframe.
- **UAT Issue Rate**: Number and severity of issues reported during User Acceptance Testing.

- **Production Deployment Success Rate**: Frequency of successful, error-free deployments to production.
- **Team Satisfaction**: Assessed via retrospective feedback, surveys, and open discussions.

12. The Importance of Your Contribution

Each of us plays a vital role in our team's success and the quality of our work.

The Power Platforms team is known for understanding site needs and delivering innovative and efficient solutions. When everyone shares ideas, communicates openly, and supports one another, we solve problems faster and more effectively. Every perspective matter and helps us make better decisions. Working together ensures our results reflect both our individual technical strengths and the shared commitment to excellence that we are now known for.